DATA 612 · Statistical Programming in R · Week 1

1.2 R Packages and the Tidyverse

Installing packages, loading them, the tidyverse, and the pipe.

1. What Is a Package?

R already contains many functions:

```
mean()
sum()
sqrt()
seq()
data.frame()
```

These functions come with **Base R** and the standard R installation.

However, not every useful function is included in Base R.

R can be extended using **packages**.

> A **package** is a collection of functions, data sets, documentation, and other tools designed for particular tasks.

Thousands of R packages are available for statistics, visualization, machine learning, data manipulation, and many other applications.

---

2. Installing a Package

Before using a package for the first time, we usually need to **install** it.

For example:

```
install.packages("ggplot2")
```

The package name must be inside quotation marks:

```
install.packages("ggplot2")
```

not:

```
install.packages(ggplot2)
```

Installation downloads the package and places it on your computer.

Usually, you install a package only once.

Think of installing an R package like installing an app on your computer.

---

3. Loading a Package

Installing a package does **not** automatically make its functions available in the current R session.

We load an installed package using:

```
library(ggplot2)
```

Notice the difference:

```
install.packages("ggplot2")   # Install
library(ggplot2)              # Load
```

A useful rule to remember is:

> **Install once; load each new R session.**

For example, you may install `ggplot2` once:

```
install.packages("ggplot2")
```

Then, whenever you start a new R session and want to use it:

```
library(ggplot2)
```

---

4. What Is an R Session?

An **R session** is the period during which R is currently running.

When you close and restart R or RStudio, you begin a new session.

Packages that you loaded previously are generally not automatically loaded into the new session.

Therefore, your script might begin with:

```
library(ggplot2)
library(dplyr)
```

but you should **not normally put** this in a script that runs every time:

```
install.packages("ggplot2")
```

Installation is something you do when needed, not every time the script runs.

---

5. Using a Function from a Package

Suppose we load `ggplot2`:

```
library(ggplot2)
```

We can now use functions provided by that package:

```
ggplot()
```

Another way to use a function is:

```
ggplot2::ggplot()
```

The notation:

```
package::function()
```

means:

> "Use this function specifically from this package."

For example:

```
dplyr::filter()
```

means:

> Use the `filter()` function from the `dplyr` package.

This can be useful because different packages can sometimes contain functions with the same name.

---

6. Getting Help

R has a built-in help system.

To read the documentation for a function:

```
?mean
```

or:

```
help(mean)
```

For a function from a package:

```
?ggplot2::ggplot
```

You can also search the help system:

```
help.search("mean")
```

A shortcut is:

```
??mean
```

Therefore:

```
?function      → documentation for a function
??topic        → search R help for a topic
```

---

7. What Packages Are Currently Loaded?

You can see information about the current R session using:

```
sessionInfo()
```

You can also see the current search path:

```
search()
```

For example, after:

```
library(ggplot2)
```

try:

```
search()
```

and notice that `ggplot2` appears in the search path.

---

8. What Packages Are Installed?

To see installed packages:

```
installed.packages()
```

This returns considerable information.

To see just their names:

```
rownames(installed.packages())
```

To check whether a particular package is available:

```
requireNamespace("ggplot2", quietly = TRUE)
```

---

9. What Is the Tidyverse?

In this course, we will learn both **Base R** and the **tidyverse**.

The tidyverse is a collection of R packages designed to work together for data science.

We can install it using:

```
install.packages("tidyverse")
```

and load it using:

```
library(tidyverse)
```

Loading `tidyverse` loads several important packages.

Some of the most commonly used are:

Package	Main purpose
`ggplot2`	Data visualization
`dplyr`	Data manipulation
`tidyr`	Reshaping and tidying data
`readr`	Reading data files
`tibble`	Modern data frames
`stringr`	Working with strings
`forcats`	Working with categorical data/factors
`purrr`	Functional programming and iteration

We will encounter these packages gradually throughout the course.

---

10. Base R vs. Tidyverse

Often, the same task can be performed using either Base R or tidyverse tools.

For example, suppose:

```
students <- data.frame(
  name = c("John", "Sara", "David"),
  grade = c(75, 92, 85)
)
```

Using **Base R**, we could select students with grades above 80:

```
students[students$grade > 80, ]
```

Using **dplyr**, we could write:

```
library(dplyr)

filter(students, grade > 80)
```

Both approaches are valid.

Throughout this course, we will learn how to work with data using both approaches.

---

11. The Pipe

One important idea in modern R and the tidyverse is the **pipe**.

Suppose:

```
x <- c(10, 20, 30, 40)
```

We could calculate its mean using:

```
mean(x)
```

Base R provides the pipe:

```
x |> mean()
```

You can read this approximately as:

> Take `x`, **then** calculate its mean.

The tidyverse historically popularized another pipe:

```
x %>% mean()
```

The `%>%` pipe is provided by `magrittr` and is commonly encountered in tidyverse code.

In this course, you may encounter both:

```
|>      # Base R pipe

%>%     # traditional tidyverse pipe
```

The pipe becomes especially useful when several operations are performed in sequence.

For example:

```
students |>
  subset(grade > 80) |>
  head()
```

Later, we will use pipes extensively with tidyverse functions.

---

12. A Typical R Workflow

A simple R project might follow this pattern:

Step 1 — Install required packages

Usually done only once:

```
install.packages("tidyverse")
```

Step 2 — Load packages

Done when beginning a new R session:

```
library(tidyverse)
```

Step 3 — Import or create data

We can create a small data set in code:

```
students <- data.frame(
  name = c("John", "Sara", "David"),
  grade = c(75, 92, 85)
)
```

We can also read a file. Download students.csv and place it in your working directory. Then:

```
students <- read.csv("students.csv")
```

With the tidyverse, an equivalent approach is:

```
students <- readr::read_csv("students.csv")
```

`read.csv()` is Base R. `read_csv()` comes from the `readr` package.

Note **1.3** is the full treatment: relative paths, delimiters, `read.table()` / `read_csv()`, inspecting the import, and exporting.

Step 4 — Explore the data

```
head(students)
str(students)
summary(students)
View(students)
```

Step 5 — Manipulate the data

```
students |>
  dplyr::filter(grade > 80)
```

Step 6 — Analyze or visualize the data

For example:

```
mean(students$grade)
```

or later:

```
ggplot(students, aes(x = name, y = grade)) +
  geom_col()
```

---

13. Important Package Commands to Remember

```
install.packages("package")
```

Install a package.

```
library(package)
```

Load an installed package.

```
package::function()
```

Use a particular **function from a particular package**.

```
?function
```

Read a function's **documentation**.

```
??topic
```

Search R's help system.

```
installed.packages()
```

See the packages that are **installed**.

```
sessionInfo()
```

See information about the current **R session**.

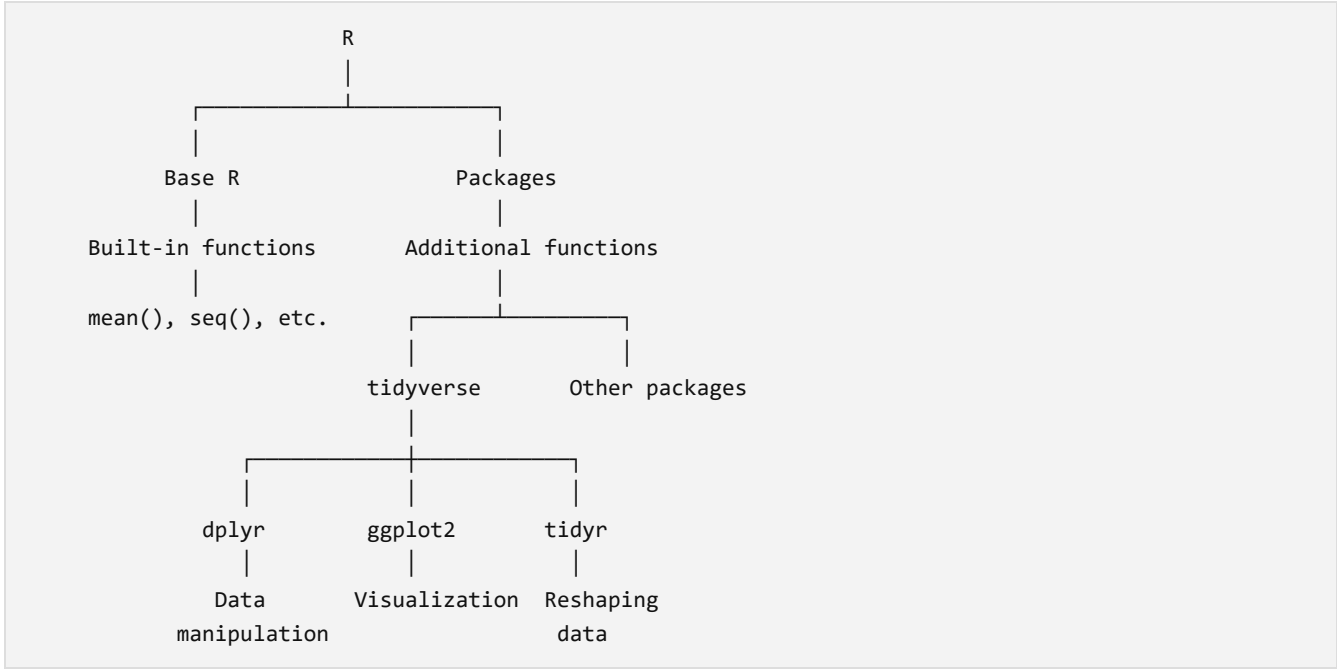
```
search()
```

See packages currently attached to the **search path**.

---

14. The Big Picture

A useful way to think about the R ecosystem is:



The central distinction to remember is:

> **R is the programming language. A package extends R by providing additional functions and tools.**

And:

> **Base R and tidyverse are not two different programming languages. The tidyverse is a collection of packages that we use within R.**

For packages, remember the three most important commands:

```
install.packages("tidyverse")   # Get it
library(tidyverse)              # Load it
dplyr::filter()                 # Use a specific function
```